

SQS, SNS, and EventBridge

Queues · Topics · Fan-Out · Event Bus · FIFO · Decoupling · ShopFlow Events

AWS Tutorial · ShopFlow

10-01

Why Messaging? Decoupling ShopFlow Services

■ Async Messaging with SQS, SNS, and EventBridge

When ShopFlow processes an order, the checkout service should not wait for the email service, inventory service, and analytics service to all finish before responding to the customer. Messaging decouples these: checkout publishes an event and returns instantly; other services consume it at their own pace.

Example: Without messaging: checkout calls email-svc, inventory-svc, analytics-svc in sequence — 800ms total. If email-svc is slow, the customer waits. With SQS/SNS: checkout publishes OrderPlaced event — 5ms. Other services process asynchronously. Customer checkout completes in 5ms.

Service	Model	Best For	ShopFlow Use Case
SQS Standard	Queue (point-to-point)	One consumer processes each message	Order notifications queue; decoupling services
SQS FIFO	Queue (ordered, exactly-once)	Message order matters; no duplicates	Inventory updates (atomic)
SNS	Topic (pub/sub fan-out)	One message delivered to multiple subscribers	OrderPlaced event to Lambda + SMS Lambda + Analytics
EventBridge	Event bus (rule-based routing)	Route events to different targets based on content	ShopFlow domain events to other services

10-02

Create an SQS Queue — Console Walkthrough

SQS is the backbone of ShopFlow async processing. Let us create the order notifications queue with a Dead Letter Queue (DLQ) for failed messages.

■ Create SQS Queue with DLQ

1. AWS Console -> search "SQS" -> click Simple Queue Service
2. Click "Create queue"
3. Type: Standard (not FIFO — we want maximum throughput)
4. Name: shopflow-order-notifications
5. Visibility timeout: 180 seconds (= 6x Lambda timeout of 30s)
6. Message retention period: 4 days (default)
7. Scroll to Dead-letter queue section -> Enable
8. Create DLQ first: click "Create queue" in a new tab -> name: shopflow-order-notifications-dlq
9. Back on main queue: DLQ ARN: paste shopflow-order-notifications-dlq ARN
10. Maximum receives: 3 (message moves to DLQ after 3 failed delivery attempts)
11. Click "Create queue"

SQS Core Concepts — Visibility Timeout, DLQ, and Ordering

10-03	SQS Core Concepts — Visibility Timeout, DLQ, and Ordering
Three SQS concepts trip up beginners most often: visibility timeout (why the same message appears twice), DLQ (where failed messages go), and Standard vs FIFO (ordering guarantees).	
Concept	How It Works
Visibility timeout	When a consumer reads a message, it becomes invisible to other consumers for a set duration (30s default). If not deleted, it reappears.
Dead Letter Queue	After maxReceiveCount failed attempts (message reappears), it moves to a DLQ for investigation.
Standard queue	At-least-once delivery, best-effort ordering. Rare duplicates possible.
FIFO queue	Exactly-once processing, strict first-in-first-out ordering.
Long polling	Consumer waits up to 20s for a message instead of short polling (1s).

Concept	How It Works	ShopFlow Setting
Visibility timeout	When a consumer reads a message, it becomes invisible to other consumers for a duration. If the processing fails, the message is re-queued.	visibilityTimeout(30s). Duration: 30s
Dead Letter Queue	After maxReceiveCount failed attempts (message read successfully but not processed), the message is moved to a dead letter queue.	maxReceiveCount(6). Sequence: 4 days
Standard queue	At-least-once delivery, best-effort ordering. Rare duplicates possible in high throughput scenarios.	At-least-once delivery, best-effort ordering. Rare duplicates possible in high throughput scenarios.
FIFO queue	Exactly-once processing, strict first-in-first-out ordering.	Exactly-once processing, strict first-in-first-out ordering.
Long polling	Consumer waits up to 20s for a message instead of checking frequently.	Consumer waits up to 20s for a message instead of checking frequently.

SQS in CDK — ShopFlow Queues

10-04 SQS in CDK — ShopFlow Queues

CDK creates all ShopFlow SQS queues with consistent configuration: encryption, DLQs, and appropriate visibility timeouts.

```
// CDK: ShopFlow SQS Queues // 1. Dead Letter Queue (create first – needed by main queues) Queue
orderNotificationsDlq = Queue.Builder.create(this, "OrderNotificationsDLQ")
.queueName("shopflow-order-notifications-dlq") .retentionPeriod(Duration.days(14))
.encryptionMasterKey(Key.fromLookup(this, "KmsKey", ...)) .build(); // 2. Standard Queue –
order notifications Queue orderNotificationsQueue = Queue.Builder.create(this,
"OrderNotifications") .queueName("shopflow-order-notifications")
.visibilityTimeout(Duration.seconds(180)) // 6x Lambda timeout
.retentionPeriod(Duration.days(4)) .deadLetterQueue(DeadLetterQueue.builder()
.queue(orderNotificationsDlq) .maxReceiveCount(3) .build())
.encryption(QueueEncryption.KMS_MANAGED) .build(); // 3. FIFO Queue – inventory updates (order
matters per product) Queue inventoryUpdatesDlq = Queue.Builder.create(this, "InventoryDLQ")
.queueName("shopflow-inventory-updates-dlq.fifo") .fifo(true).build(); Queue inventoryQueue =
Queue.Builder.create(this, "InventoryUpdates") .queueName("shopflow-inventory-updates.fifo")
.fifo(true) .contentBasedDeduplication(true) // auto dedup by message content hash
.visibilityTimeout(Duration.seconds(60)) .deadLetterQueue(DeadLetterQueue.builder()
.queue(inventoryUpdatesDlq) .maxReceiveCount(3) .build()) .build(); // 4. Grant Lambda permission
to consume from queue orderNotificationsQueue.grantConsumeMessages(orderNotificationFn);
inventoryQueue.grantConsumeMessages(inventoryUpdateFn);
```

```
// CDK: ShopFlow SQS Queues // 1. Dead Letter Queue (create first – needed by main queues) Queue
orderNotificationsDlq = Queue.Builder.create(this, "OrderNotificationsDLQ")
.queueName("shopflow-order-notifications-dlq") .retentionPeriod(Duration.days(14))
.encryptedMasterKey(Key.fromLookup(this, "KmsKey", ...)) .build(); // 2. Standard Queue –
order notifications Queue orderNotificationsQueue = Queue.Builder.create(this,
"OrderNotifications") .queueName("shopflow-order-notifications")
.visibilityTimeout(Duration.seconds(180)) // 6x Lambda timeout
.retentionPeriod(Duration.days(4)) .deadLetterQueue(DeadLetterQueue.builder()
.queue(orderNotificationsDlq) .maxReceiveCount(3) .build())
.encryptedMasterKey(Key.fromLookup(this, "KmsKey", ...)) .build(); // 3. FIFO Queue – inventory updates (order
matters per product) Queue inventoryUpdatesDlq = Queue.Builder.create(this, "InventoryDLQ")
.queueName("shopflow-inventory-updates-dlq.fifo") .fifo(true).build(); Queue inventoryQueue =
Queue.Builder.create(this, "InventoryUpdates") .queueName("shopflow-inventory-updates.fifo")
.fifo(true) .contentBasedDeduplication(true) // auto dedup by message content hash
.visibilityTimeout(Duration.seconds(60)) .deadLetterQueue(DeadLetterQueue.builder()
.queue(inventoryUpdatesDlq) .maxReceiveCount(3) .build()) .build(); // 4. Grant Lambda permission
to consume from queue orderNotificationsQueue.grantConsumeMessages(orderNotificationFn);
inventoryQueue.grantConsumeMessages(inventoryUpdateFn);
```

■ Amazon SNS (Simple Notification Service)

SNS is a pub/sub messaging service. You publish one message to an SNS topic and SNS delivers it simultaneously to all subscribers — which can be SQS queues, Lambda functions, email addresses, SMS, or HTTP endpoints. One publish, many deliveries.

Example: ShopFlow: OrderService publishes OrderPlaced to SNS topic. Subscribers: (1) SQS queue -> Lambda sends email confirmation, (2) SQS queue -> Lambda sends SMS, (3) SQS queue -> Lambda updates analytics, (4) SQS queue -> Lambda updates search index. One publish fans out to 4 consumers.

■ Create SNS Topic and Subscribe SQS Queue

1. AWS Console -> search "SNS" -> click Simple Notification Service
2. Click "Create topic"
3. Type: Standard (not FIFO)
4. Name: shopflow-order-events
5. Access policy: leave default (topic owner + publisher IAM role)
6. Click "Create topic"
7. On the topic page: click "Create subscription"
8. Protocol: Amazon SQS
9. Endpoint: paste ARN of shopflow-order-notifications queue
10. Click "Create subscription"
11. Repeat for each subscriber queue (email, SMS, analytics, search)
12. Grant SQS permission: SQS queue -> Access policy -> allow sns.amazonaws.com to SendMessage

The SNS -> SQS fan-out pattern is the backbone of ShopFlow event-driven architecture. Each service subscribes via its own SQS queue — providing buffering, retries, and independent scaling.

```
// CDK: SNS -> SQS Fan-Out for ShopFlow Order Events Topic
orderEventsTopic = Topic.Builder.create(this, "OrderEvents")
    .topicName("shopflow-order-events")
    .build(); // Fan-out: SNS topic -> 4 SQS queues -> 4 Lambda functions
// 1. Email notification
orderEventsTopic.addSubscription(new SqsSubscription(
    emailQueue, SqsSubscriptionProps.builder()
    // Filter: only CONFIRMED orders trigger email (not PENDING)
    .filterPolicy(Map.of("orderStatus", SubscriptionFilter.stringFilter(
        StringConditions.builder().allowList(List.of("CONFIRMED", "SHIPPED", "DELIVERED"))
        .build()))).rawMessageDelivery(true) // skip SNS envelope; SQS gets raw payload
    .build())); // 2. Analytics (all order events, no filter)
orderEventsTopic.addSubscription(new SqsSubscription(analyticsQueue));
// 3. Search index update (only when product details change)
orderEventsTopic.addSubscription(new SqsSubscription(
    searchIndexQueue, SqsSubscriptionProps.builder()
    .filterPolicy(Map.of("eventType", SubscriptionFilter.stringFilter(
        StringConditions.builder().allowList(List.of("PRODUCT_UPDATED", "PRODUCT_CREATED"))
        .build()))).build())); // Publish from OrderService:
snsClient.publish(PublishRequest.builder()
    .topicArn(orderEventsTopicArn)
    .message(objectMapper.writeValueAsString(orderEvent))
    .messageAttributes(Map.of("orderStatus", MessageAttributeValue.builder()
        .dataType("String").stringValue("CONFIRMED").build(), "eventType",
        MessageAttributeValue.builder().dataType("String").stringValue("ORDER_PLACED").build()))
    .build());
```

■ Amazon EventBridge

EventBridge is an event bus with powerful rule-based routing. Instead of every service knowing about every subscriber, services publish events to the bus and EventBridge routes them to targets based on event content. It also integrates with 200+ AWS services and SaaS providers as event sources.

Example: ShopFlow: inventory-svc publishes InventoryLow event to EventBridge. Rule 1 matches events where quantity < 10 AND category = "electronics" -> triggers PurchaseOrderLambda to auto-reorder. Rule 2 matches quantity < 50 -> sends alert to ops-team SNS topic.

Feature	SNS	EventBridge
Event filtering	Message attributes (limited)	Full JSON content matching (nested fields, prefix, suffix, numeric range, etc.)
Targets per rule	N/A — all subscribers get all messages (target subscription)	5 (target subscription) invoke Lambda, SQS, SNS, Step Functions
Schema registry	No	Yes — discovers and stores event schemas; generates code bindings
Cross-account	Complex (topic policies)	Native cross-account via event bus permissions
Replay	No	Yes — archive events and replay to fix bugs or test new consumers
SaaS integration	No	Yes — Shopify, Zendesk, Datadog, GitHub, etc. publish directly
Cost	\$0.50/million notifications	\$1.00/million events

EventBridge rules filter events using pattern matching on the event JSON. CDK makes rule creation readable — the pattern is a Java Map that mirrors the JSON structure of events you want to match.

```
// CDK: EventBridge Rules for ShopFlow EventBus
shopflowBus = EventBus.Builder.create(this, "ShopFlowBus")
    .eventBusName("shopflow-events").build(); // Rule 1: Low inventory alert
// (quantity < 50, any category)
Rule lowInventoryRule = Rule.Builder.create(this, "LowInventory")
    .eventBus(shopflowBus)
    .ruleName("shopflow-low-inventory-alert")
    .description("Alert when product inventory drops below threshold")
    .eventPattern(EventPattern.builder()
        .source(List.of("com.shopflow.inventory"))
        .detailType(List.of("InventoryUpdated"))
        .detail(Map.of("quantity", List.of(Map.of( // numeric range match "numeric", List.of("<", 50))))))
        .build())
    .build(); // Route to SNS alert AND SQS for auto-reorder Lambda
lowInventoryRule.addTarget(new SnsTopic(opsAlertTopic));
lowInventoryRule.addTarget(new SqsQueue(reorderQueue, SqsQueueProps.builder()
    .message(RuleTargetInput.fromEventPath("$.detail")) // pass only detail
    .build())
    .build()); // Rule 2: New user registered -> trigger welcome email
Rule newUserRule = Rule.Builder.create(this, "NewUser")
    .eventBus(shopflowBus)
    .eventPattern(EventPattern.builder()
        .source(List.of("com.shopflow.auth"))
        .detailType(List.of("UserRegistered"))
        .build())
    .build();
newUserRule.addTarget(new LambdaFunction(welcomeEmailFn)); // Publish event from Java service
eventBridgeClient.putEvents(PutEventsRequest.builder()
    .entries(List.of(PutEventsRequestEntry.builder()
        .eventBusName("shopflow-events")
        .source("com.shopflow.inventory")
        .detailType("InventoryUpdated")
        .detail(objectMapper.writeValueAsString(inventoryEvent))
        .build())
    .build())
    .build());
```

The three messaging services overlap in capability, which confuses beginners. This decision tree maps ShopFlow use cases to the right service.

INFO Decision Tree

Start here: (1) Do you need exactly-one consumer to process the message? -> SQS. (2) Do you need fan-out to multiple consumers with the same message? -> SNS (simple) or EventBridge (content-based routing / cross-account / schema registry). (3) Is the event schema complex and routing logic content-dependent? -> EventBridge. (4) Do you need to receive events from a SaaS product (Shopify, GitHub, etc.)? -> EventBridge.

Use Case	Service	Reason
Process one order notification per inventory item	SQS Standard	Point-to-point; one Lambda consumes each message
Deduct inventory in purchase order	SQS FIFO	Ordered per product ID; no duplicate deductions
Send email + SMS + analytics for every order	SNS or SQS fan-out	One publish; multiple consumers; each queue is independent
Route payment events only to finance team	EventBridge rule	Content-based filter on eventType=PAYMENT_CAPTURED
Receive Shopify inventory sync events	EventBridge SaaS integrations	Shopify is a native EventBridge partner source
Replay last 7 days of events for new service	EventBridge Archive + SQS	SQS have no replay; EventBridge archives events
Alert ops when any service emits ERROR	EventBridge rule on destination ARN	Single rule ERROR to multiple targets without SNS topic per service

10-10

FIFO Queues — Ordering and Deduplication

ShopFlow inventory management requires strict ordering: if two customers buy the last unit simultaneously, the first purchase should succeed and the second should fail. SQS FIFO guarantees per-MessageGroupId ordering to prevent race conditions.

```
// Java: Send FIFO inventory update (ordered per product)
sqsClient.sendMessage(SendMessageRequest.builder() .queueUrl(inventoryFifoQueueUrl)
    .messageBody(objectMapper.writeValueAsString(inventoryUpdate)) // MessageGroupId = productId:
    // all messages for same product // are processed in FIFO order by the same Lambda invocation
    .messageGroupId("product-" + productId) // MessageDeduplicationId prevents exact duplicate
    // messages // within 5-minute deduplication window .messageDeduplicationId(orderId + "-" +
    // productId + "-deduct") .build()); // Lambda receives messages for productId=ABC in order: //
    // msg1: deduct 2 units (order #101) -> stock: 10 -> 8 // msg2: deduct 3 units (order #102) ->
    // stock: 8 -> 5 // msg3: deduct 6 units (order #103) -> stock: 5 -> FAIL (only 5 left) // Without
    // FIFO, msg3 might process before msg2 and incorrectly succeed
```

FIFO Setting	Value	Purpose
MessageGroupId	productId (e.g. "product-SKU-12345")	All messages with same GroupId processed in order by one consumer
MessageDeduplicationId	orderId + productId	Deduplicates exact-same messages within 5-minute window
contentBasedDeduplication	true (CDK setting)	Auto-generates deduplication ID from message content hash
Max throughput	3,000 msg/s with batching	300 per GroupId per second; sufficient for ShopFlow (100K orders/day)

10-11

EventBridge Pipes — Connect Services Without Lambda

EventBridge Pipes connect a source (SQS, DynamoDB Streams, Kinesis) directly to a target (Lambda, Step Functions, SQS, SNS) with optional filtering and transformation — no custom integration Lambda needed. This reduces boilerplate and cost.

```
// CDK: EventBridge Pipe (DynamoDB Stream -> EventBridge Bus) // Replaces a custom Lambda that
// reads DDB streams and puts events on bus
CfnPipe pipe = CfnPipe.Builder.create(this,
    "ProductChangePipe") .name("shopflow-product-changes") .roleArn(pipeRole.getRoleArn()) //
    // Source: DynamoDB Streams on products table .source(productsTable.getTableStreamArn())
    .sourceParameters(CfnPipe.PipeSourceParametersProperty.builder() .dynamoDbStreamParameters(
        CfnPipe.PipeSourceDynamoDbStreamParametersProperty.builder() .startingPosition("LATEST")
        .batchSize(10) .build()) // Filter: only INSERT and MODIFY events (not DELETE)
    .filterCriteria(CfnPipe.FilterCriteriaProperty.builder()
        .filters(List.of(CfnPipe.FilterProperty.builder() .pattern("{\"eventName":
```

```
["INSERT","MODIFY"]}) .build())) .build()) .build()) // Target: EventBridge bus (rules route to search index, etc.) .target(shopflowBus.getEventBusArn()) .build();
```

INFO Pipes vs Custom Lambda

Before EventBridge Pipes, connecting DynamoDB Streams to EventBridge required a custom Lambda: read stream -> parse events -> publish to EventBridge. Pipes do this with zero custom code. Savings: one Lambda function eliminated per source-target pair. ShopFlow eliminates 3 integration Lambdas using Pipes: DDB Products -> EventBridge, DDB Orders -> EventBridge, Kinesis clickstream -> SQS.

10-12

Monitoring SQS and SNS — Key Metrics and Alarms

Queue depth and message age are the two most important SQS metrics. A growing queue means consumers are falling behind. An old message means a consumer is stuck or a DLQ is filling up.

■ aws > CloudWatch > Metrics > SQS > shopflow-order-notifications

SQS Metrics Dashboard

■ Queue metrics — last 1 hour

- ApproximateNumberOfMessagesVisible: 247 (current queue depth)
- ApproximateAgeOfOldestMessage: 42 seconds (messages processing in near real-time)
- NumberOfMessagesSent: 1,842 (messages published this hour)
- NumberOfMessagesDeleted: 1,595 (successfully processed this hour)

■ Active CloudWatch Alarms

- ALARM: shopflow-order-notifications-dlq-not-empty — DLQ depth > 0
 - OK: shopflow-order-notifications-age — Oldest message < 5 minutes
 - OK: shopflow-order-notifications-depth — Queue depth < 10,000
- DLQ alarm triggered: 3 messages failed 3 delivery attempts — check Lambda logs

Metric	Alarm Threshold	Action When Triggered
ApproximateNumberOfMessagesVisible	> 10,000 for 5 minutes	Lambda concurrency too low or Lambda failing; increase concurrency
ApproximateAgeOfOldestMessage	> 300 seconds (5 minutes)	Processing falling behind; oldest order unprocessed for 5 minutes
DLQ: NumberOfMessagesVisible	> 0	Failed message; immediate investigation; check Lambda logs
SNS: NumberOfNotificationsFailed	> 10 per minute	SNS cannot deliver to subscriber (SQS queue full, Lambda throttled)

10-13

ShopFlow Complete Event Architecture

Here is the complete messaging topology for ShopFlow. Each service owns its outbound events; it publishes to EventBridge. Consumers subscribe via SQS queues for buffering and independent scaling.

ShopFlow Event-Driven Architecture

Event Publishers

order-svc: OrderPlaced, OrderCancelled, OrderShipped -> EventBridge

inventory-svc: InventoryUpdated, InventoryLow -> EventBridge

auth-svc: UserRegistered, UserDeleted -> EventBridge

payment-svc: PaymentCaptured, PaymentFailed -> EventBridge

EventBridge Rules (shopflow-events bus)

OrderPlaced -> [email-queue, sms-queue, analytics-queue, search-queue]

InventoryLow -> [ops-alert SNS, reorder-queue]

PaymentFailed -> [fraud-review-queue, customer-support-queue]

SQS Queues -> Lambda Consumers

email-queue -> OrderNotificationLambda (email via SES)

analytics-queue -> AnalyticsLambda (DynamoDB aggregations)

reorder-queue -> AutoReorderLambda (ERP API call)

inventory-updates.fifo -> InventoryUpdateLambda (ordered stock deduction)

10-14

Hands-On Lab — Build the Order Event Pipeline

Build a mini ShopFlow order event pipeline: API Gateway -> Lambda (order-svc) -> SNS -> SQS -> Lambda (notification-svc). Time estimate: 60 minutes.

■ Lab: Order Event Pipeline

1. Create SNS topic: shopflow-order-events-lab
2. Create SQS queue: shopflow-email-notifications-lab (Standard, 180s visibility timeout)
3. Subscribe SQS queue to SNS topic (Protocol: Amazon SQS)
4. Fix SQS access policy: allow sns.amazonaws.com to SendMessage to queue ARN
5. Create Lambda: shopflow-notification-processor (Python 3.12, 256MB)
6. Handler: parse SQS event -> print "Sending email to: {email} for order: {orderId}"
7. Add SQS trigger to Lambda: select shopflow-email-notifications-lab, batch size=5
8. Create Lambda: shopflow-order-publisher (Python 3.12, 256MB)
9. Handler: publish message to SNS topic with orderData payload
10. Test: invoke shopflow-order-publisher with test payload
11. Verify: shopflow-notification-processor CloudWatch logs show email message
12. Cleanup: delete both Lambdas, SQS queue, SNS topic

Without SNS message filtering, every subscriber receives every message and must filter in their own code. SNS filter policies do this at the topic level — subscribers only receive messages matching their filter, reducing processing cost and Lambda invocations.

```
// CDK: SNS subscription with filter policy // email-queue: only CONFIRMED and SHIPPED orders
(not PENDING) orderEventsTopic.addSubscription(new SqsSubscription( emailQueue,
SqsSubscriptionProps.builder() .filterPolicy(Map.of( "orderStatus",
SubscriptionFilter.stringFilter( StringConditions.builder() .allowlist(List.of("CONFIRMED",
"SHIPPED", "DELIVERED")) .build()), "orderTotal", SubscriptionFilter.numericFilter(
NumericConditions.builder() .greaterThan(0) // exclude $0 test orders .build()))
.rawMessageDelivery(true) // no SNS envelope in SQS message body .build())); // vip-queue: only
VIP customers (orderTag contains "VIP") orderEventsTopic.addSubscription(new SqsSubscription(
vipQueue, SqsSubscriptionProps.builder() .filterPolicy(Map.of( "customerTier",
SubscriptionFilter.stringFilter( StringConditions.builder() .allowlist(List.of("VIP",
"PLATINUM")) .build())) .build()));
```

Filter Type	Example Pattern	Matches
String allowlist	{"status":["CONFIRMED","SHIPPED"]}	Messages where status is CONFIRMED or SHIPPED
String prefix	{"email":["prefix":"admin@"]}	Messages where email starts with "admin@"
Numeric range	{"total":["numeric":[">",100]]}	Messages where total > 100
Exists	{"promoCode":["exists":true]}	Messages that include the promoCode attribute
Anything-but	{"status":["anything-but":"CANCELLED"]}	All statuses except CANCELLED

SQS max message size is 256 KB. Product catalog updates with full descriptions, images metadata, and attributes can exceed this. The SQS Extended Client Library transparently stores large payloads in S3 and sends only the S3 reference in SQS.

```
// SQS Extended Client for large messages (> 256KB) // Add dependency:
software.amazon.payloadoffloading:payloadoffloading-common S3Client s3 = S3Client.create();
PayloadStorageConfiguration config = new PayloadStorageConfiguration()
.withPayloadSupportEnabled(s3, "shopflow-sqs-large-payloads") .withAlwaysThroughS3(false) //
only use S3 when > sizeThreshold .withPayloadSizeThreshold(200 * 1024); // 200 KB threshold
SqsClient extendedSqsClient = SqsClient.builder()
.overrideConfiguration(ClientOverrideConfiguration.builder() .addExecutionInterceptor( new
ExtendedClientConfiguration(config)) .build()) .build(); // Sending: if payload > 200KB,
transparently stored in S3 extendedSqsClient.sendMessage(SendMessageRequest.builder()
.queueUrl(productCatalogQueueUrl) .messageBody(largeProductCatalogJson) // e.g., 500KB JSON
.build()); // SQS message body: pointer to s3://shopflow-sqs-large-payloads/uuid // Consumer:
extendedSqsClient automatically retrieves from S3
```

Question	Answer
Can I lose messages if SQS or SNS is down?	SQS stores messages redundantly across multiple AZs within a region. Standard queue durability is 99.9999999999%.
How do I prevent the same SQS message being processed twice?	SQS Standard queue guarantees at least once delivery, not exactly-once. Implement idempotency in your application.
What is the difference between SNS and EventBridge?	SNS is simpler and cheaper (\$0.50/M vs \$1.00/M). EventBridge adds content-based routing, scheduling, and integration with other AWS services.
Why is my Lambda processing the same SQS message repeatedly?	The SQS message is deleting itself. This happens when the message body is not a valid JSON or XML document.

Processing SQS messages one at a time is inefficient. With batchSize=10, Lambda processes 10 messages per invocation — 10x fewer Lambda invocations, 10x lower cost. But partial batch failures require careful handling: if 2 of 10 messages fail, you must return only the failed message IDs to avoid reprocessing the successful 8.

```
// Java Lambda: Handle SQS batch with partial failure support public class
OrderNotificationHandler implements RequestHandler { @Override public SQSBatchResponse
handleRequest(SQSEvent event, Context ctx) { List failures = new ArrayList<>(); for
(SQSEvent.SQSMessage msg : event.getRecords()) { try { OrderNotification notification =
parseNotification(msg.getBody()); sendEmail(notification); // SES API call // Success: do NOT
add to failures list; SQS will delete this message } catch (Exception e) {
ctx.getLogger().log("Failed msg " + msg.getMessageId() + ": " + e); // Failure: add to list; SQS
will make ONLY this message visible again
failures.add(SQSBatchResponse.BatchItemFailure.builder()
.itemIdentifier(msg.getMessageId()).build()); } } // Return partial batch response - SQS
deletes successful messages, // retries only failed messages return SQSBatchResponse.builder()
.batchItemFailures(failures).build(); } }
```

Batch Setting	Value	Impact
batchSize	10 (recommended for Lambda)	10 messages per invocation; 10x fewer Lambda calls vs batch size of 1
maxBatchingWindow	5 seconds (CDK)	Lambda waits up to 5s to accumulate 10 messages; reduces number of invocations
reportBatchItemFailures	true (CDK/Lambda config)	Must set true AND return SQSBatchResponse; otherwise all messages are retried
maxConcurrency	5 (set in CDK)	Limits Lambda concurrency for this trigger; prevents one queue from monopolizing Lambda

10-19

SNS Direct Delivery — SMS and Email Endpoints

SNS can deliver directly to SMS phone numbers and email addresses, without an SQS queue. ShopFlow uses this for operational alerts: when a DLQ fills up or a service goes down, SNS sends an SMS to the on-call engineer immediately.

■ Subscribe Email and SMS to SNS Alert Topic

1. SNS Console -> select shopflow-ops-alerts topic
2. Click "Create subscription"
3. Protocol: Email -> Endpoint: ops-team@shopflow.com -> Create
4. Confirm: check email inbox, click confirmation link from AWS
5. Click "Create subscription" again
6. Protocol: SMS -> Endpoint: +12025551234 (E.164 format)
7. Click "Create subscription" (SMS does not need confirmation)
8. Test: SNS topic -> Publish message -> "Test alert: DLQ has 1 message"
9. Verify: email received within 30 seconds; SMS received within 10 seconds
10. Note: SMS only works in sandbox mode for unverified numbers; request production access for outbound SMS

Subscription Type	Format	ShopFlow Use Case	Cost
Email	user@domain.com	Ops team daily digest, weekly and monthly reports	\$0.10 (AWS charges \$0 for email delivery)
SMS	E.164: +12025551234	On-call engineer alerts (DLQ, high severity errors, etc.)	\$0.00645 per SMS (US) / \$0.014 in Europe
SQS	Queue ARN	Service-to-service messaging with SQS (SQS charges separately)	\$0.00045 per message
Lambda	Function ARN	Direct Lambda invocation (no SQS queue needed)	\$0.0001667 per invocation
HTTP/S	HTTPS endpoint	Webhook to external services (ShopFlow handles delivery)	\$0.0001667 per invocation

EventBridge Scheduler is separate from EventBridge Rules. Scheduler supports one-time future schedules (send promotional email on Black Friday at 9 AM) and flexible time windows (run within a 15-minute window to avoid thundering herd).

```
// CDK: EventBridge Scheduler for Black Friday promo email // Runs once at a specific time – not
repeating
Schedule blackFridaySchedule = Schedule.Builder.create(this, "BlackFridayPromo")
    .scheduleName("shopflow-black-friday-promo-2024") .description("Send Black Friday promotional
email to all subscribers") .schedule(ScheduleExpression.at(
Instant.parse("2024-11-29T09:00:00Z"))) // one-time: Black Friday 9AM UTC
    .flexibleTimeWindow(FlexibleTimeWindow.builder() .mode(FlexibleTimeWindowMode.FLEXIBLE)
    .maximumWindowInMinutes(10) // run within 10-min window to stagger load .build()) .target(new
LambdaInvoke(promoEmailFn, LambdaInvokeProps.builder()
    .input(ScheduleTargetInput.fromObject(Map.of( "campaign", "black-friday-2024",
"discountPercent", 30))) .build())) .build(); // One-time schedules are auto-deleted after
execution // Recurring schedule (every 15 minutes): Schedule.Builder.create(this,
"Cachewarmer") .schedule(ScheduleExpression.rate(Duration.minutes(15))) .target(new
LambdaInvoke(cachewarmerFn, ...)) .build();
```

INFO Scheduler vs EventBridge Rules for Cron

EventBridge Rules (schedule source) are the original cron mechanism — free, simple, granularity to 1-minute. EventBridge Scheduler (\$0.01/million invocations above 14M/month free) adds: time zones, flexible windows, one-time schedules, and per-schedule retry policies. For ShopFlow: use EventBridge Rules for recurring cron jobs; use Scheduler for one-time promotional campaigns.

At 1M MAU with ShopFlow typical order and event volumes, messaging costs are minimal — typically under \$20/month. Understanding the pricing model prevents surprise bills.

Service	Pricing	ShopFlow Monthly Volume	Monthly Cost
SQS Standard	\$0.40 per million API calls (first 1M free)	20M API calls (5M orders x 4 calls each) + 1M paid calls (all check)	\$7.60 (\$0.40 x 19M paid calls)
SQS FIFO	\$0.50 per million API calls	2M API calls (500K inventory updates x 4 calls each) + 1M paid calls	\$0.50 (\$0.50 x 1M paid calls)
SNS	\$0.50 per million notifications (first 1M free)	5M notifications (1.5M orders x 4 subscribers) + 1M paid	\$2.50 (\$0.50 x 5M paid)
SNS SMS	\$0.00645 per SMS in US	100K SMS alerts/month	\$0.645
EventBridge	\$1.00 per million events (first 1M free)	2M events/month (all domain events)	\$2.00 (\$1.00 x 2M paid)
TOTAL	—	—	~\$13.25/month

TIP Free Tier is Generous

SQS: first 1 million requests per month free forever (not just 12 months). SNS: first 1 million publishes free forever. EventBridge: first 1 million custom events free forever. ShopFlow messaging bill at typical 1M MAU volume is roughly \$13/month — far less than equivalent point-to-point service calls would cost in latency, retry infrastructure, and engineering time.

Tracing an async order through SQS and SNS is harder than tracing a synchronous HTTP call. AWS X-Ray propagates trace context through SQS message attributes so you can see the full order journey: API -> order-svc -> SNS -> SQS -> notification Lambda in a single trace.

```
// Enable X-Ray tracing end-to-end through SQS/SNS // 1. Enable X-Ray on Lambda (CDK) Function
fn = Function.Builder.create(this, "OrderNotification") .tracing(Tracing.ACTIVE) // Lambda
sends segments to X-Ray .build(); // 2. Use X-Ray SDK in Java Lambda to create subsegments
import com.amazonaws.xray.AWSXRay; import com.amazonaws.xray.entities.Subsegment; public Void
handleRequest(SQSEvent event, Context ctx) { for (SQSEvent.SQSMessage msg : event.getRecords())
```

```
{ // X-Ray automatically propagates trace from SQS message attributes Subsegment seg =
AWSXRay.beginSubsegment("send-email"); try { sendEmail(parseNotification(msg.getBody()));
seg.putAnnotation("orderId", extractOrderId(msg)); seg.putAnnotation("emailSent", true); }
catch (Exception e) { seg.addException(e); throw e; } finally { AWSXRay.endSubsegment(); } }
return null; } // 3. View trace in CloudWatch: // Console -> CloudWatch -> X-Ray traces ->
filter by traceId // See: API Gateway (2ms) -> Lambda order-svc (45ms) -> SQS publish (8ms) //
-> [async gap] -> Lambda notification (820ms total: email send 780ms)
```

10-23

SQS Delay Queues — Scheduled Future Processing

SQS delay queues make messages invisible to consumers for a configurable period (0-900 seconds) after they are sent. ShopFlow uses this for: holding order cancellation requests for 5 minutes (customer can undo), and delaying fraud review for 2 minutes after payment.

Delay Feature	Queue-Level Delay	Per-Message Delay
How to set	SQS Console -> Edit queue -> Delivery delay (Seconds)	SendMessage API: DelaySeconds parameter (overrides queue setting)
Use case	All messages in queue delayed by same amount	Individual messages delayed by different amounts
ShopFlow use	cancellation-queue: 300s delay (5-min undo window)	fraud-review: 120s per message regardless of queue setting
Visibility	Delayed messages show in ApproximateNumberOfMessagesVisible metric	Not visible to consumers; no policy required; delay enforced by queue

```
// Per-message delay: fraud review 2 minutes after payment
sqsClient.sendMessage(SendMessageRequest.builder() .queueUrl(fraudReviewQueueUrl)
.messageBody(objectMapper.writeValueAsString(paymentEvent)) .delaySeconds(120) // invisible for
2 minutes; then consumer picks up .build()); // Queue-level delay: order cancellation 5-minute
undo window // Set delivery delay = 300 on shopflow-cancellation-queue // All messages
automatically held 5 minutes before consumer can read them
```

10-24

SNS FIFO Topics — Ordered Fan-Out

Standard SNS topics do not guarantee delivery order. SNS FIFO topics (paired with SQS FIFO queues) provide strict ordering for fan-out scenarios. ShopFlow uses SNS FIFO for order status updates — all subscribers see status changes in the correct sequence.

Feature	SNS Standard	SNS FIFO
Delivery order	Best-effort (may reorder)	Strict order per MessageGroupId
Deduplication	None	5-minute dedup window (same as SQS FIFO)
Throughput	Nearly unlimited	300 publishes/second (3,000 with batching)
Subscribers	SQS, Lambda, Email, SMS, HTTP	SQS FIFO queues ONLY
Naming	Any name	Must end in .fifo
ShopFlow usage	Order events fan-out (email, analytics, search)	Order status updates (must arrive in order: PENDING->CONFIRMED)

INFO When to Use SNS FIFO

SNS FIFO has one major constraint: it can only deliver to SQS FIFO queues (not Lambda directly, not email/SMS). If any of your subscribers need direct Lambda delivery or email/SMS, use Standard SNS with application-level sequencing instead. For ShopFlow order status, all subscribers use SQS FIFO queues, so SNS FIFO is the correct choice.

10-25

EventBridge Archive and Replay — Fix Bugs Retroactively

EventBridge Archive stores all events matching a pattern for a configurable retention period. If a consumer Lambda has a bug and processes events incorrectly, you can fix the bug and replay the archived events — re-processing the last N days of events through the corrected consumer.

■ Set Up EventBridge Archive and Replay

1. EventBridge Console -> select shopflow-events bus
2. Left sidebar -> Archives -> Create archive
3. Archive name: shopflow-events-30day
4. Event source: shopflow-events (the bus)
5. Retention: 30 days
6. Filter: leave empty to archive ALL events (or add pattern to filter)
7. Click "Create"
8. --- To replay after fixing a bug ---
9. Archives -> select shopflow-events-30day -> Start replay
10. Replay name: fix-analytics-lambda-2024-11-01
11. Time range: 2024-10-15 00:00 to 2024-10-31 23:59
12. Destination: shopflow-events bus (re-publishes events to trigger rules)
13. Click "Start replay" — events re-delivered to analytics Lambda

TIP Real ShopFlow Scenario

ShopFlow analytics Lambda had a bug from Oct 15-31: it was not counting orders from mobile app users (missing `device_type=mobile` filter). After fixing the Lambda on Nov 1, we replayed 16 days of archived `OrderPlaced` events. The corrected analytics Lambda reprocessed all mobile orders, and the monthly report was accurate. Without archive, those 16 days of data would be lost forever.

10-26

Black Friday Messaging Readiness Checklist

At 10x normal traffic, messaging bottlenecks appear in unexpected places. SQS queue depth spikes, SNS delivery failures, and Lambda concurrency throttling all cascade. Prepare in advance.

Checklist Item	Normal Traffic	Black Friday Target
SQS queue depth alarm	Alert at 10,000 messages	Alert at 50,000 (10x higher; expect larger backlog)
Lambda concurrency for SQS consumers	200 (reserved)	500 — request Service Quotas increase 2 weeks early
SQS batchSize	10 messages per Lambda	Keep at 10; increase maxConcurrency from 5 to 25 to process 25 batches
SNS delivery retries	Default: 3 retries	Verify custom retry policy: immediate, 1s, 20s, 20x20s = up to 7 minutes
DLQ monitoring	Alert on > 0 messages	Alert on > 0 messages; P0 incident if DLQ grows during checkout window
EventBridge quota	10,000 events/second default	Verify via Service Quotas; increase to 50,000 if needed
Load test	Not done	Run at 3x expected peak; verify queue metrics, no throttles, DLQs stable

10-27

Event Schema Design — Structure and Versioning

Well-designed event schemas prevent breaking changes when services evolve. ShopFlow uses a standard envelope format for all events: every event has a consistent outer structure, with service-specific data in the detail field.

```
// ShopFlow Standard Event Envelope (all events follow this structure) { "specversion": "1.0",  
// CloudEvents spec version "id": "uuid-v4", // globally unique event ID "source":  
"com.shopflow.order-svc", // service that emitted event "type": "com.shopflow.order.placed", //  
event type (reverse DNS + verb) "time": "2024-11-29T14:23:45Z", // ISO-8601 UTC timestamp
```

```
"datacontenttype": "application/json", "schemaversion": "1.2", // payload schema version
"data": { "orderId": "ORD-2024-001234", "customerId": "USR-5678", "items": [...], "total":
149.99, "currency": "USD" } } // Schema versioning rules: // MINOR version bump (1.1 -> 1.2):
add NEW optional fields – backward compatible // MAJOR version bump (1.x -> 2.0): remove or
rename fields – deploy new consumer first // Never remove a field without bumping major version
– consumers break silently
```

Schema Practice	Correct Approach
Use additive changes only for minor versions	Add new optional fields; never remove or rename existing fields in minor versions
Include event ID in every message	Enables idempotency checks — consumers can detect and skip duplicate event deliveries
Register schemas in EventBridge Schema Registry	Discover event structure; generate Java code bindings with EventBridge Schema Registry
Use CloudEvents specification format	Industry-standard envelope adopted by AWS, GCP, Azure, and Knative — future-proofs y

Handling Poison Messages — DLQ Operational Patterns

A poison message is a malformed or unprocessable message that causes a consumer to fail on every delivery attempt. Without a DLQ, it blocks the queue or causes infinite retry loops. With a DLQ, it is isolated. The operational question is: what do you do with DLQ messages?

DLQ Operational Pattern	How It Works	When to Use
Fix and redrive	Fix the consumer bug that caused the failure, then use SQS Console DLQ Redrive actions to send all DLQ messages back to the original queue	Most SQS Console DLQ Redrive actions send all DLQ messages back to the original queue
Manual inspect and discard	Read each DLQ message, log/alert the business team, then delete the message	Messages are valuable business data (e.g., signed payload, highly sensitive data)
Transform and redrive	Read DLQ messages, fix the payload (add missing fields, correct invalid business data), then send messages to the original queue	Messages are valuable business data (e.g., signed payload, highly sensitive data)
Alert and escalate	DLQ alarm triggers PagerDuty page to on-call engineering business messages — every failure has business impact	Financial or regulatory business messages — every failure has business impact

INFO DLQ Redrive from Console

SQS Console -> Queues -> select DLQ -> Dead-letter queue actions -> Start DLQ redrive. Select destination: Original source queue. Choose "Redrive all messages" (or set max messages). Click "Start DLQ redrive". Progress shown in real time. Redriven messages re-enter the original queue with full visibility timeout and receive count reset to 0.

SNS HTTP Endpoints — Webhooks to External Services

SNS can deliver to HTTPS endpoints — external services, Slack webhooks, PagerDuty, or your own monitoring dashboard. ShopFlow sends operational alerts to Slack and triggers PagerDuty incidents for P0 events.

```
// CDK: Subscribe HTTPS endpoint to SNS topic // (Slack webhook, PagerDuty, or custom endpoint)
orderEventsTopic.addSubscription(new UrlSubscription(
"https://hooks.slack.com/services/T00000000/B00000000/XXXX", UrlSubscriptionProps.builder()
.protocol(SubscriptionProtocol.HTTPS) .filterPolicy(Map.of( // Only P0 events go to Slack
ops-alerts channel "severity", SubscriptionFilter.stringFilter( StringConditions.builder()
.allowlist(List.of("P0", "P1")) .build())) .deliveryPolicy(DeliveryPolicy.builder()
.healthyRetryPolicy(HealthyRetryPolicy.builder() .numRetries(3)
.minDelayTarget(Duration.seconds(20)) .maxDelayTarget(Duration.seconds(20)) .build()) .build())
.build()); // Note: SNS sends a subscription confirmation POST to the endpoint first // The
endpoint must respond with the SubscribeURL from the confirmation payload // Slack webhooks and
most SaaS endpoints handle this automatically
```

External Service	Protocol	ShopFlow Use Case	Notes
Slack	HTTPS (incoming webhooks)	ops-alerts channel: DLQ alarms, high severity events	SNS message format in Slack format; use Lambda for parsing
PagerDuty	HTTPS (events v2 API)	P0 incident creation: payment service outages	PagerDuty Events API accepts raw JSON; configure incident types
Custom dashboard	HTTPS (internal endpoint)	Real-time event count display on operational dashboard	Deploy dashboard behind ALB with HTTPS; SNS verifies certificate

ShopFlow uses separate AWS accounts for production, staging, and each major business unit (marketplace, fulfillment, analytics). EventBridge enables cross-account event delivery without complex VPC peering or shared credentials.

```
// Cross-account EventBridge: Production account publishes to Analytics account // Step 1:
Analytics account — add event bus permission for Production account
CfnEventBusPolicy.Builder.create(this, "AllowProdAccount")
    .statementId("allow-prod-account-123456789") .action("events:PutEvents")
    .principal("123456789012") // Production AWS account ID
    .eventBusName("shopflow-analytics-events") // bus in Analytics account .build(); // Step 2:
Production account — EventBridge rule routes to cross-account bus Rule orderRule =
Rule.Builder.create(this, "CrossAccountOrders") .eventBus(prodShopflowBus) // source: prod
event bus .eventPattern(EventPattern.builder() .source(List.of("com.shopflow.order-svc"))
    .detailType(List.of("OrderPlaced"))) .build()) .build(); // Target: event bus in Analytics
account (different account + same region) orderRule.addTarget(new EventBus(
    EventBus.fromEventBusArn(this, "AnalyticsBus",
        "arn:aws:events:us-east-1:987654321098:event-bus/shopflow-analytics-events"))); // Result:
OrderPlaced events automatically flow from Production to Analytics // without shared IAM
credentials, VPC peering, or custom integration code
```

This table summarizes every messaging component in ShopFlow production, with queue settings, consumer details, and operational notes. Use this as your reference during implementation and operations.

Component	Type	Config	Consumer	Volume
shopflow-order-events	SNS Standard topic	No filter on topic itself	4 SQS subscriber queues (email, SMS, analytics, ops)	500K/month
shopflow-order-notifications	SQS Standard	Visibility=180s, DLQ after 3 fails, OrderModification	OrderModificationLambda (batchSize=10)	50K/month
shopflow-inventory-updates.fifo	SQS FIFO	ContentDedup=true, Visibility=60s, FIFO	InventoryUpdateLambda (batchSize=1)	25K/month
shopflow-events	EventBridge bus	Archive 30d, 8 active rules	Varies: Lambda, SQS, SNS targets	1M events/month
shopflow-ops-alerts	SNS Standard	Email+SMS subscribers	Ops team email + on-call SMS	50 alerts/week
shopflow-cancellations	SQS Standard	Delay=300s (5min undo window), CancelOrder	CancelOrderProcessorLambda	10K/month

Operational Runbook	Action
DLQ alarm fires	1. Check Lambda CloudWatch Logs for error pattern. 2. Fix the bug. 3. Deploy fixed Lambda. 4. SQS messages in DLQ are not deleted.
Queue depth > 10,000	1. Check Lambda Errors metric — if errors: fix bug first. 2. If no errors: Lambda concurrency too low.
SNS delivery failures spike	1. Check SQS subscriber queue — is it over max message size or access policy issue? 2. Check Lambda logs.
EventBridge rule not triggering	1. EventBridge Console -> Rules -> Test event pattern with sample event JSON. 2. Check event bus permissions.

Best Practice	Anti-Pattern to Avoid
Set visibility timeout = 6x Lambda timeout to prevent message loss.	Setting visibility timeout equal to or less than Lambda timeout — causes double delivery.
Always configure a DLQ; alarm on DLQ depth > 0 for immediate action.	No DLQ: failed messages silently disappear after 3 retries with no trace or metrics.
Use SNS -> SQS fan-out pattern: buffer each consumer.	SNS to Lambda directly for high-volume events — Lambda throttles propagation.
Use SQS FIFO with MessageGroupId for operations that require idempotency (e.g., inventory deductions).	SQS Standard for idempotency: inventory deductions can oversell stock with retries.
Use EventBridge for content-based routing and cross-service orchestration.	Direct service-to-service calls for async operations — tight coupling, cascading failures.
Set rawMessageDelivery=true on SNS->SQS subscription.	Default rawMessageDelivery=false: consumer must unwrap SNS envelope.

